

Utilities Component - Configuration-as-Factory Pattern

C

Console

default_sttings : dict
is_debug : bool
is_verbose
prefix : Optional[str]
show_timestamps : bool
verbose : bool

dbg(message: str) -> None
error(exception: Exception, message: Optional[str]) -> None
log(message: str) -> None
plog(obj: Any, title: Optional[str]) -> None
print_instructions(instructions: str) -> None
print_schema(schema_json: str) -> None
set_debug(is_debug: bool) -> 'Console'
set_prefix() -> 'Console'
set_timestamp_display(show_timestamps: bool) -> 'Console'
set_verbose(verbose: bool) -> 'Console'
unset_prefix() -> 'Console'
warn(message: str) -> None
with_prefix() -> 'Console'

Structured logging via Rich

- Debug/verbose modes
- Prefix management
- Timestamp display
- Schema and Config pretty-printing

C

PathConfig

root : Optional[Path]

data : Annotated[Path, Field(default='.data/SmartAIs-Recorded-Data')]
env_file : Annotated[Path, Field(default='.env')]
convert_to_path(v: str | Path, info: ValidationInfo) -> Path
get_api_key(key_name: Union[Literal['GOOGLE_API_KEY'], str]) -> Optional[str]

C

NoTarget

setup_target(config: 'BaseConfig') -> None

C

SingletonConfig

model_config

Singleton pattern for global paths

- Environment file management
- API key resolution
- Data directory configuration

Configuration-as-Factory Pattern

- Base class for each component's config
- Type-safe object instantiation
- Automatic validation and dependency injection
- Generic target creation via setup_target()

C

BaseConfig

model_config
propagated_fields : Optional[Dict[str, Any]]
protected_fields : Optional[List[str]]
target : Optional[Callable[['BaseConfig[TargetType]', TargetType]]]

inspect(show_docs: bool) -> None
setup_target() -> TargetType